

You are looking at the documentation for the latest official release.

Waveshare RP2350

The [RP2350 SoC](#) ⁴³ by Raspberry Pi Ltd. is a small sized and low-cost 32-bit dual ARM Cortex-M33 and dual 32-bit Hazard3 RISC-V (RV32IMAC+) microcontroller and predestined for versatile board designs. The Waveshare RP2350 board series based on this microcontroller offers a wide range with different scaling factors, in size, features and interfaces for communication, input and output.

Supported Boards

Hardware

RP2350-CAN

The RP2350-CAN is a [pico sized](#) CAN bus focused development board. The on-board USB-C connector makes it compatible to future standards. It comes with an on-board SPI/CAN controller, fully supports CAN V2.0B specification with communication rate up to 1 Mbps, together with an CAN transceiver and higher current high-efficiency DC-DC buck-boost converter. The castellated module allows soldering direct to carrier boards.

Features and Resources

Printed Circuit Board

5V/1.1A

3.3V/1.8A

3.3V(OUT)

3.3V(EN/PS)

150MHz

4MB

520KB

RST GRN 

L/H CAN

USB-C CAN 

UF2

26 GPIO

16 PWM

3 ADC

2 I2C

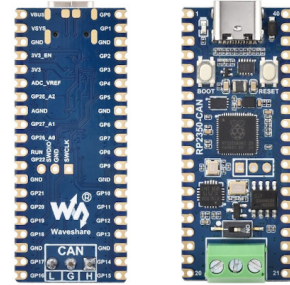
2 SPI

2 UART

- Dual core Arm Cortex-M33 processor running up to 150MHz
- **520KB** on-chip SRAM
- **4MB** on-board QSPI flash with XIP capabilities
- USB 1.1 controller (host/device)
- On-board **USB-C connector**
- On-board **DC-DC buck-boost converter with 2.0A switch**
- On-board **RESET** button
- On-board **BOOT** button
- On-board **LED**
- **26 GPIO** pins via **edge pinout**
- **2 UART** peripherals
- **2 I2C** controllers
- **2 SPI** controllers
- **16 PWM** channels
- **3 ADC** analog inputs
- 12 Programmable I/O (PIO) state machines for custom peripherals
- 1 Watchdog timer peripheral
- 1 Temperature sensor on-chip

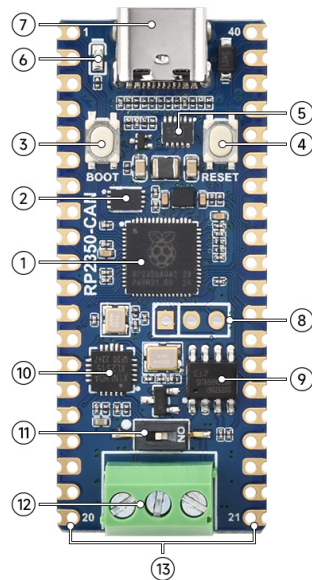
Design Data

- [RP2350-CAN Schematic](#) ²¹
- [RP2350-CAN Demo](#) ²² in C/Pico-SDK and Python/Thonny IDE



Positions

RP2350-CAN



- | | |
|---------------------------------|-------------------------------|
| 1. RP2350A | 8. Debug pins |
| 2. On-board flash memory | Serial Wire Debug (SWD) |
| 4MB NOR-Flash | 9. SIT65HVD230DR |
| P25Q32SH-UXH-IR | (SN65HVD230) |
| (W25Q32JV) | CAN transceiver |
| 3. BOOT button | 10. XL2515QF20 |
| press it when | (MCP2515) |
| resetting to enter | CAN controller |
| download mode | 11. Terminal resistor |
| 4. RESET button | selection |
| 5. MP28164 | enable/disable 120R |
| 1.8A DC-DC buck- | CAN matching resistor |
| boost converter, | 12. CAN screw terminal |
| power switch | 13. Pinout |
| 6. LED monochrome | compatible with |
| user LED (not power | Raspberry Pi Pico |
| indicator) | |
| 7. USB Type-C | |
| connector | |
| Supports USB1.1 | |
| hosts and slave | |
| devices | |

Data Sheets

- | | |
|---|--|
| • RP2350 SoC ⁴³ | • P25Q32SH Datasheet V1.8 (2023/10) ³⁹ |
| • RP2350 Datasheet ⁴⁴ | • P25Q32SH Datasheet V1.7 (2023/08) ⁴⁰ |
| • Hardware design with RP2350 ⁴⁵ | • W25Q32JV Datasheet ⁴² |
| • P25Q32SH ³⁶ (W25Q32JV ⁴¹) | • XL2515QF20 ³⁴ (MCP2515 ²⁷) |
| • P25Q32SH Datasheet V2.0 (2025/07) ³⁷ | • XL2515QF20 Datasheet ³⁵ |
| • P25Q32SH Datasheet V1.9 (2024/07) ³⁸ | • MCP2515 Datasheet ²⁸ |

- [MCP2515 Silicon Errata](#) ²⁹
 - [SIT65HVD230](#) ³⁰ ([SN65HVD230](#) ³²)
 - [SIT65HVD230 Datasheet](#) ³¹
 - [SN65HVD230 Datasheet](#) ³³
- [MP28164](#) ²⁴
 - [MP28164 Datasheet](#) ²⁵
 - [MP28164 Reliability Report](#) ²⁶

Flash partitions

QSPI NOR-Flash	code_partition	storage_partition
primary 4MB	1MB	3MB
secondary N/A	–	–

Pinouts

The peripherals of the [RP2350 SoC](#) ⁴³ can be routed to various pins on the board. The configuration of these routes can be modified through [DTS](#). Please refer to the datasheet to see the possible routings for each peripheral. The default assignments for the various Waveshare RP2350 boards are defined below separately in a single tab.

RP2350-CAN

External pin mapping on the [RP2350-CAN](#) is identical to the original [Raspberry Pi Pico](#) board, but note that internal RP2350A GPIO lines 23 and 24 are routed to the voltage regulator and USB connector for SMPS (MP28164) power saving modes and USB VBUS sense. GPIO line 25 is routed to the on-board user LED and GPIO line 29 will be used for VSYS/3 voltage monitoring per default.

In addition, GPIO lines 8 to 12 are used for control and SPI communication with the on-board SPI/CAN controller, but are still available externally in parallel.

Pin Mapping	Pinout
-------------	--------

on-edge(1-40):

8 GPIO

8 PWM

3 ADC

2 I2C

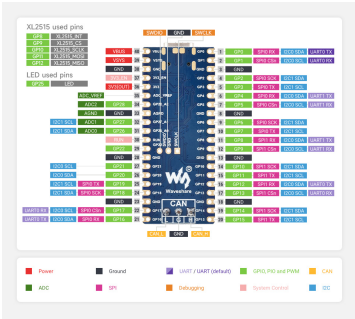
1 SPI

1 UART

on-board:

1 GRN 

1 L/H CAN



Default Zephyr Peripheral Mapping

- 1

UART0_TX

: GP0 (PWM0)
- 2

UART0_RX

: GP1 (PWM1)
- 3

GND
- 4

PIO/PWM :

GP2

PWM2
- 5

PIO/PWM :

GP3

PWM3
- 6

I2C0_SDA

: GP4 (PWM4)
- 7

I2C0_SCL

: GP5 (PWM5)
- 8

GND
- 9

PIO/PWM :

GP6

PWM6
- 10

PIO/PWM :

GP7

PWM7
- 11

GPIO8

: GP8 (PWM8)
- on-board LCD data/cmd

CAN_INT
- 12

SPI1_CSN

: GP9 (PWM9)
- on-board LCD chip select

CAN_CS
- 13

GND
- 14

SPI1_SCK

: GP10 (PWM10)
- on-board LCD i/f clock

CAN_SCLK
- 15

SPI1_TX

: GP11 (PWM11)
- on-board LCD i/f data

CAN_MOSI
- 16

SPI1_RX

: GP12 (PWM12)
- on-board LCD reset

CAN_MISO
- 17

PIO/PWM :

GP13

PWM13
- 18

GND
- 19

I2C1_SDA

: GP14 (PWM14)
- 20

I2C1_SCL

: GP15 (PWM15)
- 21

SPI0_RX

: GP16 (PWM0)
- 22

SPI0_CSN

: GP17 (PWM1)
- 23

GND
- 24

SPI0_SCK

: GP18 (PWM2)
- 25

SPI0_TX

: GP19 (PWM3)
- 26

PIO/PWM :

GP20

PWM4
- 27

PIO/PWM :

GP21

PWM5

28

GND

29

PIO/PWM : GP22

PWM6

30

RUN (RESET)

PIO/PWM : GP23 (PWM7)

on-board SMPS power save PS

PIO/PWM : GP24 (PWM8)

on-board VBUS/1.51 sense

PIO/PWM : GP25

PWM9

on-board user LED

31

ADC_CH0 : GP26 (PWM10)

32

ADC_CH1 : GP27 (PWM11)

33

GND

34

ADC_CH2 : GP28 (PWM12)

ADC_CH3 : GP29 (PWM13)

on-board VSYS/3 monitoring

35

ADC_VREF

36

3V3(OUT)

37

3V3_EN

on-board SMPS power enable

38

GND

39

VSYS

40

VBUS

- Devicetree compatible
- raspberrypi,pico-header
 - raspberrypi,pico-header-r3

Supported Features

Similar to the [Raspberry Pi Pico 2](#) the Waveshare RP2350 board configuration supports the following hardware features:

Peripheral	Kconfig option	Devicetree compatible	Zephyr API
PINCTRL	CONFIG_PINCTRL	raspberrypi,pico-pinctrl	Pin Control API

Peripheral	Kconfig option	Devicetree compatible	Zephyr API
GPIO	CONFIG_GPIO	raspberrypi,pico-gpio	General-Purpose Input/Output (GPIO)
UART	CONFIG_SERIAL	raspberrypi,pico-uart	Universal Asynchronous Receiver-Transmitter (UART)
UDC (USB Device Controller)	CONFIG_USB_DEVICE_STACK_NEXT	raspberrypi,pico-usbd	USB device support APIs
I2C	CONFIG_I2C	raspberrypi,pico-i2c	Inter-Integrated Circuit (I2C) Bus
SPI	CONFIG_SPI	raspberrypi,pico-spi	Serial Peripheral Interface (SPI) Bus
PWM	CONFIG_PWM	raspberrypi,pico-pwm	Pulse Width Modulation (PWM)
ADC	CONFIG_ADC	raspberrypi,pico-adc	Analog-to-Digital Converter (ADC)
Temperature (Sensor)	CONFIG_SENSOR	raspberrypi,pico-temp	Sensors
Timer (Counter)	CONFIG_COUNTER	raspberrypi,pico-timer	Counter
Watchdog Timer (WDT)	CONFIG_WATCHDOG	raspberrypi,pico-watchdog	Watchdog
Flash	CONFIG_FLASH	raspberrypi,pico-flash-controller	Flash and Flash map
PIO	CONFIG_PIO_RPI_PICO	raspberrypi,pico-pio	N/A
UART (PIO)	CONFIG_SERIAL	raspberrypi,pico-uart-pio	Universal Asynchronous Receiver-Transmitter (UART)
SPI (PIO)	CONFIG_SPI	raspberrypi,pico-spi-pio	Serial Peripheral Interface (SPI) Bus
WS2812 (PIO)	CONFIG_LED_STRIP	worldsemi,ws2812-rpi-pico-pio	N/A
DMA	CONFIG_DMA	raspberrypi,pico-dma	Direct Memory Access (DMA)
HWINFO	CONFIG_HWINFO	N/A	Hardware Information
VREG	CONFIG_REGULATOR	raspberrypi,core-supply-regulator (!)	Regulators
RESET	CONFIG_RESET	raspberrypi,pico-reset	Reset Controller

Peripheral	Kconfig option	Devicetree compatible	Zephyr API
CLOCK	CONFIG_CLOCK_CONTROL	raspberrypi,pico- clock-controller raspberrypi,pico- clock	Clock Control
NVIC	N/A	arm,v8m-nvic	Nested Vector Interrupts Controller
SYSTICK	N/A	arm,armv8m-systick	

(!) POWMAN with VREG on RP2350 not yet supported by Zephyr.

See section **Peripherals RP2350** in upstream issue: <https://github.com/zephyrproject-rtos/zephyr/issues/53810>

Other hardware features are not currently supported by Zephyr. The default configuration can be found in the different Kconfig files:

- [boards/waveshare/rp2350/waveshare_rp2350_can_rp2350a_m33_defconfig](#)

Board Configurations

The Waveshare RP2350 boards can be configured for the following different use cases.

RP2350-CAN

west build -b waveshare_rp2350_can/rp2350a/m33

Use the serial port UART0 on edge header as Zephyr console and for the shell.

west build -b waveshare_rp2350_can/rp2350a/m33 -S usb-console

Use the native USB device port with CDC-ACM as Zephyr console and for the shell.

Connections and IOs

The [Waveshare wiki](#) ²⁰ has detailed information about board connections. Download the different schematics or datasheets as linked above per board for more details. The pinout diagrams can also be found there.

System Clock

The [RP2350A](#) ⁴³ MCU is configured to use the 12MHz external crystal with the on-chip PLL generating the 150MHz system clock. The internal AHB and APB units are set up in the same way

as the upstream [Raspberry Pi Pico C/C++ SDK](#) ¹⁵ libraries.

GPIO (PWM) Ports

The [RP2350A](#) ⁴³ MCU has 1 GPIO cell which covers all I/O pads and 8 PWM function unit each with 2 channels beside a dedicated Timer unit. On the [RP2350-CAN](#), PWM4 channel B is available on the on-board user LED. But the PWM operation is not enable by default. Only if `CONFIG_PWM_RPI_PICO` is enabled then the first user LED is driven by PWM4CHB instead of by GPIO. All channels of PWM0 until PWM7 are available on the `Raspberry Pi Pico` header.

ADC/TS Ports

The [RP2350A](#) ⁴³ MCU has 1 ADC with 4 channels and an additional fifth channel for the on-chip temperature sensor (TS). The ADC channels 0-2 are available on the `Raspberry Pi Pico` header. On the [RP2350-CAN](#), ADC channel 3 will be used for internal on-board voltage monitoring.

The external voltage reference `ADC_VREF` can be used optional for the ADC and is only available on the `Raspberry Pi Pico` header.

SPI Port

The [RP2350A](#) ⁴³ MCU has 2 SPIs. To the edge connectors SPI0 is connect to external devices over GP19 (MOSI), GP16 (MISO), GP18 (SCK), and GP17 (CSn) on the `Raspberry Pi Pico` header.

I2C Port

The [RP2350A](#) ⁴³ MCU has 2 I2Cs. To the edge connectors I2C0 and I2C1 is connect to external devices over GP4 (I2C0_SDA), GP5 (I2C0_SCL), GP14 (I2C1_SDA), and GP15 (I2C1_SCL) on the `Raspberry Pi Pico` header.

Serial Port

The [RP2350A](#) ⁴³ MCU has 2 UARTs. One of the UARTs (UART0) is connected to external devices over GP0 (TX) and GP1 (RX) on the `Raspberry Pi Pico` header and is the Zephyr console.

USB Device Port

The [RP2350A](#) ⁴³ MCU has a (native) USB device port that can be used to communicate with a host PC. See the [USB](#) sample applications for more, such as the [USB CDC ACM UART sample](#) which sets up a virtual serial port that echos characters back to the host PC. As an alternative to the default Zephyr console on serial port the Bridle [USB Console Snippet \(usb-console\)](#) can be used to enable [CDC ACM](#) and switch the console to USB

```

USB device idVendor=2e8a, idProduct=000a, bcdDevice= 4.04
USB device strings: Mfr=1, Product=2, SerialNumber=3
Product: RP2350-CAN (CDC ACM)
Manufacturer: Waveshare (Raspberry Pi)
SerialNumber: B46993A480CF94B1

```

To integrate specific USB device functions that do not follow a USB standard class, the following alternate identifier numbers are available for the various Waveshare RP2040 and RP2350 boards according to the [Raspberry Pi USB product ID list](#) ¹³:

RP2040

0x101F	:	RP2040-Zero
0x1020	:	RP2040-Plus
0x1021	:	RP2040-LCD-0.96
0x1039	:	RP2040-LCD-1.28
0x103A	:	RP2040-One
0x1044	:	Power Management HAT (B)
0x1055	:	RP2040-ETH
0x1056	:	RP2040-Geek
0x1057	:	RP2040-Touch-LCD-1.28
0x1083	:	RP2040-PiZero
0x1084	:	RP2040-Tiny
0x1085	:	RP2040-Matrix
0x1086	:	RP2040-BLE
0x1087	:	PICO-Cam-A

RP2350

0x10B0	:	RP2350-Zero
0x10B1	:	RP2350-Plus
0x10B2	:	RP2350-Tiny
0x10B3	:	RP2350-LCD-1.28
0x10B4	:	RP2350-Touch-LCD-1.28
0x10B5	:	RP2350-One
0x10B6	:	RP2350-Geek
0x10B7	:	RP2350-LCD-0.96
0x10C3	:	RP2350-ETH

Programmable I/O (PIO)

The [RP2350 SoC](#) ⁴³ comes with three PIO peripherals. These are three simple co-processors that are designed for I/O operations. The PIOs run a custom instruction set, generated from a custom assembly language. PIO programs are assembled using **pioasm**, a tool provided by Raspberry Pi. Further information can be found in the [Raspberry Pi Pico C/C++ SDK](#) ¹⁵ document, section with title “Using PIOASM, the PIO Assembler”.

Zephyr does not (currently) assemble PIO programs. Rather, they should be manually assembled and embedded in source code. An example of how this is done can be found at [drivers/serial/uart_rpi_pico_pio.c](#) or [drivers/spi/spi_rpi_pico_pio.c](#).

Programming and Debugging

Flashing

Using UF2

If you don't have an SWD adapter, you can flash the Waveshare RP2350 boards with a UF2 file. By default, building an app for this board will generate a `build/zephyr/zephyr.uf2` file. If the board is powered on with the `BOOTSEL` button pressed, it will appear on the host as a mass storage device:

```
USB device idVendor=2e8a, idProduct=000f, bcdDevice= 1.00
USB device strings: Mfr=1, Product=2, SerialNumber=0
Product: RP2350 Boot
Manufacturer: Raspberry Pi
SerialNumber: A0231978E046
```

The UF2 file should be drag-and-dropped or copied on command line to the device, which will then flash the Waveshare RP2350 board.

Each [RP2350 SoC](#)⁴³ ships the [UF2 compatible](#)⁹ bootloader [pico-bootrom-rp2350](#)¹⁶, a native support in silicon. The full source for the RP2350 bootrom at [pico-bootrom-rp2350](#)¹⁶ includes versions A2, A3 and A4 of the bootrom, which correspond to the same silicon revisions, respectively.

Note that every time you build a program for the RP2350, the Pico SDK selects and creates an appropriate image definition and/or partition table block with attributes and precedes it in the firmware. Further information can be found in the [RP2350 Datasheet](#)⁴⁴, sections with title “*Bootrom*” and “*Processor Controlled Boot Sequence*”.

Using SEGGER JLink

You can flash the Waveshare RP2350 boards with a SEGGER JLink debug probe as described in [Building, Flashing and Debugging](#).

Here is an example of building and flashing the [Blinky](#) application.

```
west build -b waveshare_rp2350_can/rp2350a/m33 -p -d build/waveshare_rp2350 zephyr/samples/basic/blinky
west flash -r jlink -d build/waveshare_rp2350
```

Using OpenOCD

To use [PicoProbe](#) ¹⁷ or [Raspberry Pi Debug Probe](#) ¹⁸, you must configure **udev**. Create a file in `/etc/udev.rules.d` with any name, and write the line below:

```
ATTRS{idVendor}=="2e8a", ATTRS{idProduct}=="0004", MODE="660", GROUP="plugdev", TAG+="uaccess"
ATTRS{idVendor}=="2e8a", ATTRS{idProduct}=="000c", MODE="660", GROUP="plugdev", TAG+="uaccess"
```

This example is valid for the case that the user joins to `plugdev` groups.

The [RP2350-CAN](#) has an SWD interface that can be used to program and debug the on board RP2350. This interface can be utilized by OpenOCD. To use it with the RP2350, OpenOCD version 0.12.0 or later is needed. If you are using a Debian based system (including RaspberryPi OS, Ubuntu, and more), using the [pico_setup.sh](#) ¹⁹ script is a convenient way to set up the forked version of OpenOCD. Depending on the interface used (such as JLink), you might need to checkout to a branch that supports this interface, before proceeding. Build and install OpenOCD as described in the README.

Here is an example of building and flashing the [Blinky](#) application.

```
west build -b waveshare_rp2350_can/rp2350a/m33 -p -d build/waveshare_rp2350 zephyr/samples/basic/blinky -- \
-DOPENOCD=/usr/local/bin/openocd \
-DOPENOCD_DEFAULT_PATH=/usr/local/share/openocd/scripts \
-DWAVESHARE_RP2350_DEBUG_ADAPTER=picoprobe
west flash -r openocd -d build/waveshare_rp2350
```

Set the environment variables **OPENOCD** to `/usr/local/bin/openocd` and **OPENOCD_DEFAULT_PATH** to `/usr/local/share/openocd/scripts`. This should work with the OpenOCD that was installed with the default configuration. This configuration also works with an environment that is set up by the [pico_setup.sh](#) ¹⁹ script.

WAVESHARE_RP2350_DEBUG_ADAPTER specifies what debug adapter is used for debugging. If **WAVESHARE_RP2350_DEBUG_ADAPTER** was not assigned, *cmsis-dap* is used by default. The other supported adapters are *picoprobe*, *raspberrypi-swd*, *jlink* and *blackmagicprobe*. How to connect *picoprobe* and *raspberrypi-swd* is described in [Getting Started Guide with Raspberry Pi Pico](#) ¹⁴. Any other SWD debug adapter maybe also work with this configuration. The value of **WAVESHARE_RP2350_DEBUG_ADAPTER** is cached, so it can be omitted from **west flash** and **west debug** if it was previously set while running **west build**.

WAVESHARE_RP2350_DEBUG_ADAPTER is used in an argument to OpenOCD as `"source [find interface/${WAVESHARE_RP2350_DEBUG_ADAPTER}.cfg]"`. Thus,

WAVESHARE_RP2350_DEBUG_ADAPTER needs to be assigned the file name of the debug

adapter.

You can also flash the board with the following command that directly calls OpenOCD (assuming a SEGGER JLink adapter is used):

```
$ openocd -f interface/jlink.cfg \
  -c 'transport select swd' \
  -f target/rp2350.cfg \
  -c "adapter speed 2000" \
  -c 'targets rp2350.core0' \
  -c 'program path/to/zephyr.elf verify reset exit'
```

Debugging

The SWD interface can also be used to debug the board. To achieve this, you can either use SEGGER JLink or OpenOCD.

Using SEGGER JLink

Use a SEGGER JLink debug probe and follow the instruction in [Building, Flashing and Debugging](#).

Using OpenOCD

Install OpenOCD as described for flashing the board.

Here is an example for debugging the [Blinky](#) application.

```
west build -b waveshare_rp2350_can/rp2350a/m33 -p -d build/waveshare_rp2350 zephyr/samples/
basic/blinky -- \
-DOPENOCD=/usr/local/bin/openocd \
-DOPENOCD_DEFAULT_PATH=/usr/local/share/openocd/scripts \
-DWAVESHARE_RP2350_DEBUG_ADAPTER=raspberrypi-swd
west debug -d build/waveshare_rp2350
```

As with flashing, you can specify the debug adapter by specifying **WAVESHARE_RP2350_DEBUG_ADAPTER** at **west build** time. No needs to specify it at **west debug** time.

You can also debug with OpenOCD and gdb launching from command-line. Run the following command:

```
$ openocd -f interface/jlink.cfg \
  -c 'transport select swd' \
  -f target/rp2350.cfg \
  -c "adapter speed 2000" \
  -c 'targets rp2350.core0'
```

On another terminal, run:

```
$ gdb-multiarch
```

Inside gdb, run:

```
(gdb) tar ext :3333  
(gdb) file path/to/zephyr.elf
```

You can then start debugging the board.

More Samples

LED Blinky and Fade

RP2350-CAN

Green User LED Blinky by GPIO

See also Zephyr sample: [Blinky](#).

On ARM Cortex-M33

```
west build -b waveshare_rp2350_can/rp2350a/m33 -p -d build/waveshare_rp2350 zephyr/  
samples/basic/blinky  
west flash -r uf2 -d build/waveshare_rp2350
```

On Hazard3 RISC-V (RV32IMAC+)

```
west build -b waveshare_rp2350_can/rp2350a/hazard3 -p -d build/waveshare_rp2350 zephyr/  
samples/basic/blinky  
west flash -r uf2 -d build/waveshare_rp2350
```

Green User LED Blinky by PWM

See also Zephyr sample: [PWM Blinky](#).

On ARM Cortex-M33

```
west build -b waveshare_rp2350_can/rp2350a/m33 -p -d build/waveshare_rp2350 zephyr/  
samples/basic/blinky_pwm  
west flash -r uf2 -d build/waveshare_rp2350
```

On Hazard3 RISC-V (RV32IMAC+)

```
west build -b waveshare_rp2350_can/rp2350a/hazard3 -p -d build/waveshare_rp2350 zephyr/  
samples/basic/blinky_pwm  
west flash -r uf2 -d build/waveshare_rp2350
```

Green User LED Fade by PWM

See also Zephyr sample: [Fade LED](#).

On ARM Cortex-M33

```
west build -b waveshare_rp2350_can/rp2350a/m33 -p -d build/waveshare_rp2350 zephyr/  
samples/basic/fade_led  
west flash -r uf2 -d build/waveshare_rp2350
```

On Hazard3 RISC-V (RV32IMAC+)

```
west build -b waveshare_rp2350_can/rp2350a/hazard3 -p -d build/waveshare_rp2350 zephyr/  
samples/basic/fade_led  
west flash -r uf2 -d build/waveshare_rp2350
```

Hello Shell with USB-CDC/ACM Console

RP2350-CAN

Hello Shell

See also Bridle sample: [Hello Shell](#).

On ARM Cortex-M33

```
west build -b waveshare_rp2350_can/rp2350a/m33 -p -S "usb-console" -d build/  
waveshare_rp2350_bridle/samples/helloshell  
west flash -r uf2 -d build/waveshare_rp2350
```

On Hazard3 RISC-V (RV32IMAC+)

```
west build -b waveshare_rp2350_can/rp2350a/hazard3 -p -S "usb-console" -d build/  
waveshare_rp2350_bridle/samples/helloshell  
west flash -r uf2 -d build/waveshare_rp2350
```

Simple test execution on target

(text in bold is a command input)

Basics

LED

PWM

GPIO

ADC

Timer

Flash access

I2C

Sensors

CAN

```
uart:~$ hello -h  
hello - say hello  
uart:~$ hello  
Hello from shell.
```

```
uart:~$ hwinfo devid  
Length: 8  
ID: 0xb46993a480cf94b1  
  
uart:~$ kernel version  
Zephyr version 4.4.0  
  
uart:~$ bridle version  
Bridle version 4.4.0  
  
uart:~$ bridle version long  
Bridle version 4.4.0.0  
  
uart:~$ bridle info  
Zephyr: 4.4.0  
Bridle: 4.4.0
```

```
uart:~$ device list
devices:
- clock-controller@40010000 (READY)
  DT node labels: clocks
- reset-controller@40020000 (READY)
  DT node labels: reset
- snippet_cdc_acm_console_uart (READY)
  DT node labels: snippet_cdc_acm_console_uart
- uart@40070000 (READY)
  DT node labels: uart0 rpipico_serial pico_serial
- watchdog@400d8000 (READY)
  DT node labels: wdt0
- timer@400b8000 (READY)
  DT node labels: timer1
- timer@400b0000 (READY)
  DT node labels: timer0
- dma@50000000 (READY)
  DT node labels: dma
- gpio-port@0 (READY)
  DT node labels: gpio0 gpio0_lo
- usbd@50110000 (READY)
  DT node labels: usbd zephyr_udc0
- adc@400a0000 (READY)
  DT node labels: adc
- flash-controller@400d0000 (READY)
  DT node labels: qmi
- i2c@40090000 (READY)
  DT node labels: i2c0 rpipico_i2c0 rpipico_i2c pico_i2c0 pico_i2c
- pwm@400a8000 (READY)
  DT node labels: pwm
- spi@40088000 (READY)
  DT node labels: spi1 rpipico_spi1 pico_spi1
- spi@40080000 (READY)
  DT node labels: spi0 rpipico_spi0 rpipico_spi pico_spi0 pico_spi
- xl2515@0 (READY)
  DT node labels: xl2515_can0 can0
- pwm_leds (READY)
  DT node labels: pwm_leds
- dietemp (READY)
  DT node labels: die_temp
```

```
uart:~$ history
[ 0] history
[ 1] device list
[ 2] bridle info
[ 3] bridle version long
[ 4] bridle version
[ 5] kernel version
[ 6] hwinfo devid
[ 7] hello
[ 8] hello -h
```

RP2350-CAN

See also how to [Building and Running](#) the CANnectivity USB to CAN application firmware. The USB [Device Firmware Upgrade](#) (DFU) via the [MCUboot](#) ⁵ bootloader is not and will never be supported. The UF2 bootloader is perfect for firmware updates.

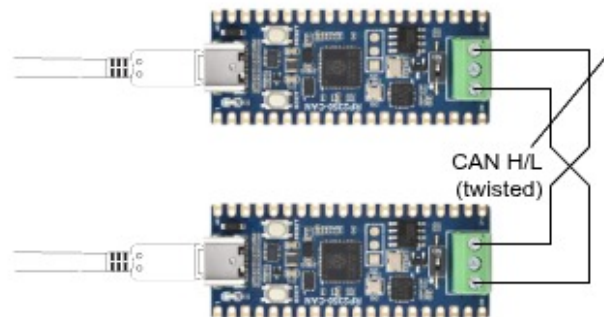
On ARM Cortex-M33

```
west build -b waveshare_rp2350_can/rp2350a/m33 -p -d build/waveshare_rp2350 connectivity/
app -- -DCONF_FILE="prj_usbd_next_release.conf"
west flash -r uf2 -d build/waveshare_rp2350
```

On Hazard3 RISC-V (RV32IMAC+)

```
west build -b waveshare_rp2350_can/rp2350a/hazard3 -p -d build/waveshare_rp2350
connectivity/app -- -DCONF_FILE="prj_usbd_next_release.conf"
west flash -r uf2 -d build/waveshare_rp2350
```

To verify the CANnectivity application is running, plug in two devices to your Linux development host, check the Linux kernel messages and use the [can-utils](#) ³ for testing.



Simple test execution on Linux host

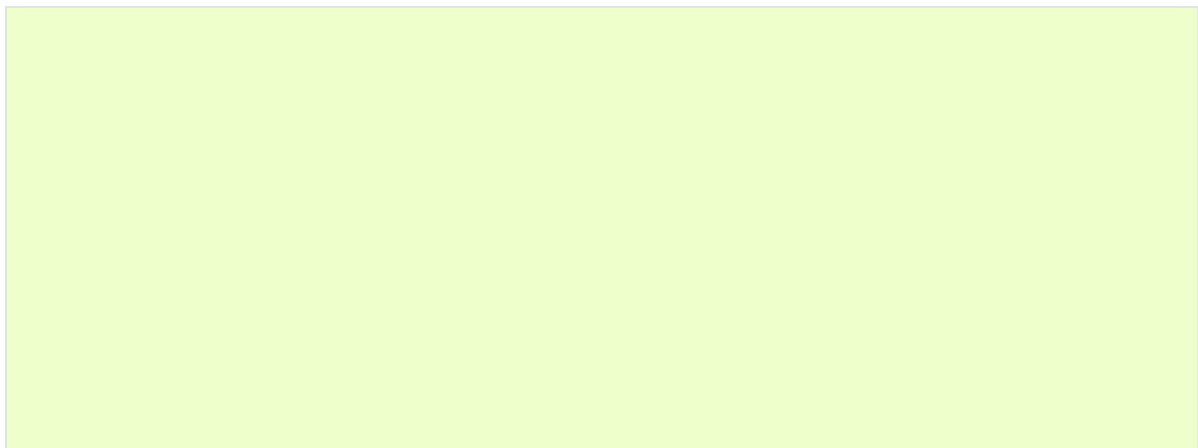
(text in **bold** is a command input)

Linux kernel messages

SocketCAN interface setup

SocketCAN user space

Assuming the [CANnectivity udev rule](#) ² installed the **dmesg** output is as follow:



```
$ dmesg -w
usb 1-6.3: new full-speed USB device number 57 using xhci_hcd
usb 1-6.3: New USB device found, idVendor=1209, idProduct=ca01, bcdDevice= 1.04
usb 1-6.3: New USB device strings: Mfr=1, Product=2, SerialNumber=3
usb 1-6.3: Product: CANnectivity USB to CAN adapter
usb 1-6.3: Manufacturer: CANnectivity
usb 1-6.3: SerialNumber: B46993A480CF94B1
gs_usb 1-6.3:1.0: Configuring for 1 interfaces

usb 1-6.4: new full-speed USB device number 58 using xhci_hcd
usb 1-6.4: New USB device found, idVendor=1209, idProduct=ca01, bcdDevice= 1.04
usb 1-6.4: New USB device strings: Mfr=1, Product=2, SerialNumber=3
usb 1-6.4: Product: CANnectivity USB to CAN adapter
usb 1-6.4: Manufacturer: CANnectivity
usb 1-6.4: SerialNumber: BC6E3797335B08DC
gs_usb 1-6.4:1.0: Configuring for 1 interfaces
```

References

- [1] <https://zephyrproject.org/>
- [2] <https://raw.githubusercontent.com/CANnectivity/connectivity/refs/heads/main/99-connectivity.rules>
- [3] <https://github.com/linux-can/can-utils>
- [4] <https://www.doxygen.org/>
- [5] <https://www.trustedfirmware.org/projects/mcuboot>
- [6] (1, 2, 3, 4, 5, 6, 7) <https://www.python.org/downloads>
- [7] <https://pypi.org/project/Pillow>
- [8] (1, 2) <https://pypi.org/project/Sphinx>
- [9] <https://github.com/Microsoft/uf2#user-content-bootloaders>
- [10] <https://www.sphinx-doc.org/en/master/usage/theming.html>
- [11] (1, 2) <https://www.sphinx-doc.org/en/master/usage/restructuredtext/basics.html>
- [12] <https://sitemaps.org/protocol.html>
- [13] <https://github.com/raspberrypi/usb-pid>
- [14] <https://datasheets.raspberrypi.com/pico/getting-started-with-pico.pdf>
- [15] (1, 2) <https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf>
- [16] (1, 2) <https://github.com/raspberrypi/pico-bootrom-rp2350>
- [17] <https://github.com/raspberrypi/picoprobe>
- [18] <https://www.raspberrypi.com/products/debug-probe>
- [19] (1, 2) https://raw.githubusercontent.com/raspberrypi/pico-setup/master/pico_setup.sh

- [20] <https://www.waveshare.com/wiki>
- [21] <https://www.waveshare.com/wiki/RP2350-CAN#Documents>
- [22] https://www.waveshare.com/wiki/RP2350-CAN#Demo_2
- [23] https://wiki.seeedstudio.com/Grove-Barometer_Sensor-BMP280
- [24] <https://www.monolithicpower.com/mp28164.html>
- [25] https://www.monolithicpower.com/en/documentview/productdocument/index/version/2/document_type/Datasheet/lang/en/sku/MP28164GD-Z/document_id/1764
- [26] https://www.monolithicpower.com/en/documentview/productdocument/index/version/2/document_type/Reliability%20Report/lang/en/sku/MP28164GD-Z/document_id/6077
- [27] <https://www.microchip.com/en-us/product/mcp2515>
- [28] <https://ww1.microchip.com/downloads/aemDocuments/documents/APID/ProductDocuments/DataSheets/MCP2515-Family-Data-Sheet-DS20001801K.pdf>
- [29] <https://ww1.microchip.com/downloads/aemDocuments/documents/OTH/ProductDocuments/Errata/80000179H.pdf>
- [30] <http://en.sitcores.com/show-38-154-1.html>
- [31] <http://www.sitcores.com/uploadfile/2023/1110/20231110052516809.pdf>
- [32] <https://www.ti.com/product/SN65HVD230>
- [33] <https://www.ti.com/lit/ds/symlink/sn65hvd230.pdf>
- [34] <https://www.xinluda.com/en/CAN-Bus-controller/20250103618.html>
- [35] https://files.waveshare.com/wiki/common/XL2515_DataSheet_EN.pdf
- [36] https://www.puyasemi.com/en/h_series653/3177.html
- [37] https://www.puyasemi.com/download_path/数据手册/Flash/P25Q32SH_Datasheet_V2.0.pdf
- [38] https://www.puyasemi.com/download_path/数据手册/Flash/P25Q32SH_Datasheet_V1.9.pdf
- [39] https://www.puyasemi.com/download_path/数据手册/Flash/P25Q32SH_Datasheet_V1.8.pdf
- [40] https://www.puyasemi.com/download_path/数据手册/Flash/P25Q32SH_Datasheet_V1.7.pdf
- [41] https://www.winbond.com/hq/product/code-storage-flash-memory/serial-nor-flash/index.html?__locale=en&partNo=W25Q32JV
- [42] https://www.winbond.com/hq/support/documentation/levelOne.jsp?__locale=en&DocNo=DA00-W25Q32JV.1
- [43] (1, 2, 3, 4, 5, <https://www.raspberrypi.com/products/rp2350>
6, 7, 8, 9, 10,
11, 12)
- [44] (1, 2) <https://datasheets.raspberrypi.com/rp2350/rp2350-datasheet.pdf>
- [45] <https://datasheets.raspberrypi.com/rp2350/hardware-design-with-rp2350.pdf>